ON THIS PAGE

Start here

Setup

SECTION 1 · LLMS, FINANCE LENS

Overview

1.1 Finance words

1.2 Picking a model

1.3 Finance models & datasets

1.4 Lab 1A · Earnings JSON

1.5 Prompting, finance style

1.6 Fine-tuning in finance

1.7 Lab 1B · Knowledge cutoff

1.8 Lab 1C · Live price agent

SECTION 2 · PATTERNS, FINANCE EDITION

Overview

2.1 Financial ratios

2.2 Lab 2A · Ratio tool

2.3 Lab 2B · Working memory

2.4 Lab 2C · Episodic memory

2.5 Reasoning in finance

2.6 Lab 2E · Verified news loop

2.7 Lab 2F · Buy/Hold/Sell

2.8 Context & memory upkeep

Wrap-up & quiz

FUTURE WITH SHIVANK · HANDS-ON TRACK

AI Agents in Finance

Section 1 & Section 2 — the finance-first, code-first edition

You already know how agents, tools, memory and multi-agent systems work from our **AI Agents on AWS** module. So this page skips all of that and adds only two new things: **the finance knowledge** you need to build agents a bank would actually use, and **the code**, explained line by line in plain English.

🏦 **Your role for this module:** you have just joined the AI team at **Future with Shivank Bank** (a made-up bank). Analysts, fund managers and advisors keep walking up to your desk with requests. Every lab on this page builds one thing they asked for.

LAB 1A
Earnings-call summariser
Turns a long call transcript into a clean JSON card for fund managers.

LAB 1C
Live price agent
Answers "What is NVIDIA trading at?" with real data, not a guess.

LAB 2A
Ratio retriever
Pulls Apple's profitability, valuation and liquidity ratios from a data API.

LAB 2B + 2C
Rebalancing assistant
Remembers the chat (working memory), then learns across sessions (episodic memory).

LAB 2E
Verified news agent
Reuters only, last two days, every item with a link, checked by a second agent.

LAB 2F
Buy / Hold / Sell pipeline
Three agents pass JSON like a relay baton to produce an investment memo.

How to read the coloured boxes:

🧠 Analogy · 💰 Finance corner · ⚠️ Watch out · ⏩ Already covered · 💡 Tip · 🧪 Simulation (click it!)

Setup Before you start (3 minutes)

KEY	NEEDED FOR	COST
`OPENAI_API_KEY`	Every lab	A few cents for everything on this page
`FINANCIAL_MODELING_PREP_API_KEY`	Ratio labs (2A, 2F)	Free plan is enough
`GOOGLE_API_KEY`	Gemini comparison in Lab 1A (optional)	Free tier available
`HF_TOKEN` + a GPU	Llama comparison in Lab 1A (optional)	Free token; request access to Llama 3.1 8B Instruct on its Hugging Face page first

Terminal · install once Copy

```
pip install openai openai-agents google-genai yfinance financetoolkit chromadb llama-index transformers tor
```

Python · load your keys (top of every lab) Copy

```python
import os
# Put keys in environment variables (or Colab secrets) — never paste them into code you share
OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]
FINANCIAL_MODELING_PREP_API_KEY = os.environ["FINANCIAL_MODELING_PREP_API_KEY"]
```

💡 **WHY DO SOME LINES START WITH `AWAIT`?**

Agents wait on the internet (the LLM, a price API). The SDK lets Python *wait without freezing* — that is what `async` / `await` means. In **Jupyter or Colab** you can type `await Runner.run(...)` directly in a cell. In a normal **.py file**, either use `Runner.run_sync(...)` (no await needed) or wrap your code like this:

Python · running async code from a .py file Copy

```python
import asyncio
from agents import Agent, Runner

async def main():
    result = await Runner.run(my_agent, "Hello")   # 'await' is allowed inside an 'async def'
    print(result.final_output)

asyncio.run(main())                                # starts the async world and runs main()
```

⚠️ **TWO HABITS THAT SAVE YOU AN HOUR**

- **One fresh notebook (or kernel restart) per lab.** Several labs reuse names like `get_company_fundamentals`; an old version left in memory silently gets used.
- **Your output will differ from the samples here.** LLM answers vary run to run, and prices, ratios and news are live.

SECTION 1

LLMs through a finance lens

Same building blocks you already know — but a bank asks different questions of them: *Is this number fresh? Where does client data go? Can a fund manager's dashboard read this output without breaking?* This section answers those, with three small labs.

> **ALREADY COVERED — WE WON'T REPEAT**
>
> What an LLM or a Transformer is · how RAG retrieves, augments and generates · what makes a system "agentic" · how tool calling works · "start with the simplest thing that works" before reaching for agents. If any of these feel rusty, revisit the AWS module. Below, we only keep the **finance angle** and the **code**.

1.1 Finance words you'll meet in the labs

The labs read earnings calls and stock data. Ten words unlock almost all of it:

Ticker	A stock's short code. `AAPL` = Apple, `NVDA` = NVIDIA.
Earnings call	A quarterly call where a company's leaders explain results to investors. The *transcript* is the text of that call.
Revenue	All the money from sales. Also called the "top line".
Net income	Profit left after every cost, interest and tax. The "bottom line".
EPS	Earnings Per Share = net income ÷ number of shares. "How much profit each share earned."
Guidance	Management's own forecast: "we expect revenue to grow mid-single digits next quarter."
YoY	Year-over-year: compared with the same quarter last year.
Fiscal quarter	The company's own accounting quarter. Apple's year ends in September, so its "Q3" is April–June.
Sentiment	Is this news good, bad or neutral for the stock?
Portfolio manager (PM)	The person deciding what a fund buys and sells. Wants short, decision-ready summaries.

> **ANALOGY — THE QUARTERLY REPORT-CARD MEETING**
>
> An earnings call is a company's parent–teacher meeting. **Revenue** is all the fees the school collected, **net income** is what is left after paying every bill, **EPS** is that leftover split per family, and **guidance** is the teacher saying "here's what we expect next term". Investors listen for surprises.

1.2 Picking the right model for a finance job

In a bank, model choice comes down to two questions:

1. **How much thinking does the task need?** *Non-reasoning* models answer straight away — fast and cheap. *Reasoning* models think in hidden steps first — slower, pricier, more careful. Many APIs let you set a *reasoning effort* (none → high).
2. **How sensitive is the data?** *Closed* models (GPT, Claude, Gemini) are the most capable, but your prompt travels to the provider. *Open-weights* models (Llama, Mistral, Gemma, GPT-OSS) can run on the bank's own servers, so client data never leaves — but you need GPUs and a team to run them.

How sensitive is the data? ↑	Simple, fast task	Multi-step, deep reasoning
Confidential	**Small open model, self-hosted** Data never leaves the bank's servers. e.g. tag internal client emails by topic	**Open reasoning model, self-hosted** or an enterprise deal with data controls e.g. credit review of private loan files
Public	**Fast hosted model (or FinBERT)** Cheap, low latency, high volume. e.g. score 50k public headlines an hour	**Hosted reasoning model** Slower, pricier, more careful. e.g. compare 8 quarters of public filings

How much reasoning does the task need? →

A rule of thumb for Future with Shivank Bank: first ask "where may this data go?", then "how hard is the thinking?". Example: short intraday trade summaries → fast model; a multi-quarter financial analysis → reasoning model.

> **ANALOGY — CASHIER VS AUDITOR**
>
> A non-reasoning model is the bank cashier: counts notes fast, all day. A reasoning model is the auditor: slow, checks every step, costs more per hour. You don't send the auditor to count notes, and you don't let the cashier sign off the annual accounts.

1.3 Finance-specialised models, tools and datasets

Finance has its own family of models. Know what each one *is* — especially which ones only classify and which ones can write.

NAME	KIND	WHAT IT'S GOOD AT

FinBERT	Open · *classifier*	Labels a financial sentence as positive / negative / neutral. It cannot write a summary.
FinGPT	Open framework + models	Fine-tuned on news, social media, filings and announcements — sentiment, robo-advice, quant ideas.
InvestLM	Open · instruction-tuned	Investment Q&A; trained on CFA-exam questions, SEC filings and quant-finance discussions.
FinLlama	Open · Llama-based	Question answering over filings and reports.
FinMA	Open (PIXIU project)	Sentiment, headline classification, named-entity recognition, Q&A.
SEC-BERT	Open · BERT family	Reading SEC filings and pulling out key entities.
BloombergGPT	Closed · in-house	Trained on a huge private financial corpus; Q&A, classification, summarisation.
Assistants inside finance platforms	Closed · products	Claude for Financial Services, ChatGPT inside Excel with financial-data connectors, LSEG Workspace AI, FactSet Mercury, Thomson Reuters CoCounsel, AlphaSense search — LLMs wired into the data analysts already use.

This landscape changes every few months — treat the list as a snapshot and check leaderboards for the current state.

Datasets = the exam papers. When someone says "our model understands finance", ask *"on which benchmark?"* These four come up most often:

DATASET	WHAT IT TESTS
Financial PhraseBank	Sentiment of single news sentences (positive / negative / neutral)
FinQA	Number crunching over real financial statements, with expert step-by-step solutions
ECTSum	Short bullet-point summaries of earnings-call transcripts
FiQA	Opinion-style Q&A and sentiment about specific aspects of a company

TRY IT · 6 LINES

See a classifier in action (runs on a laptop CPU)

Python · finbert_sentiment.py

Copy

```python
from transformers import pipeline

# FinBERT is a classifier: sentence in → label out. It cannot write text.
finbert = pipeline("text-classification", model="ProsusAI/finbert")

headlines = [
    "Apple reports record-breaking revenue in Q4 despite market slowdown",
    "Tesla stock falls after earnings miss expectations",
]
for h in headlines:
    print(h, "→", finbert(h)[0])     # e.g. {'label': 'positive', 'score': 0.93}
```

Notice the speed: no prompt, no JSON — just a label and a confidence score. That is why a specialised small model often beats a giant chat model for one narrow, high-volume job.

1.4 Lab 1A · Earnings-call summariser that returns JSON

The request: "Every quarter I read twenty earnings calls. Give me each one as the same fixed card — company, quarter, revenue, EPS, guidance, highlights, risks, outlook — so it can drop straight into our dashboard."

We'll send the same request to three models: GPT-4.1-mini (closed), Gemini 2.5 Flash (closed) and Llama 3.1 8B (open). The transcript is a made-up Apple call stored in a variable called `transcript_apple_earnings_call`.

Step 1 — Write the two prompts

Python · lab_1a_earnings_json.py · step 1

Copy

```python
from openai import OpenAI
import json, os

client = OpenAI(api_key=os.environ["OPENAI_API_KEY"])

# WHO the model is + WHAT SHAPE the answer must have
system_prompt = """You are a finance analyst. Produce a concise, decision-useful summary of an
earnings call for portfolio managers. Return strict JSON with keys:
company, quarter, revenue, eps, guidance, highlights[], risks[], outlook."""

# The actual job + the data. The f"..." drops the transcript text into the {...} slot.
user_prompt = f"""Summarize the following earnings call transcript:
{transcript_apple_earnings_call}"""
```

① `system_prompt` is the job description: the role ("finance analyst"), the audience ("portfolio managers") and the **output contract** — the exact keys. `highlights[]` is a hint that this key should hold a list.

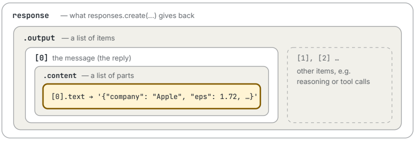
2. `user_prompt` is the task for this one call. The `f` in front of the quotes makes it an *f-string*: whatever is inside `{ }` gets replaced by that variable's value — here, the whole transcript.

Step 2 — Call the model and unpack the answer

Python · lab_1a_earnings_json.py · step 2

```python
response = client.responses.create(
    model="gpt-4.1-mini",                                    # cheap + fast: fine for summarising
    input=[{"role": "system", "content": system_prompt},
           {"role": "user",   "content": user_prompt}],
    text={"format": {"type": "json_object"}},                # JSON mode: the reply must be valid JSON
)

raw_text = response.output[0].content[0].text                # dig the text out of the response box
summary  = json.loads(raw_text)                              # JSON text  →  Python dictionary
print(json.dumps(summary, indent=2))                         # dictionary →  pretty, indented text
```


response — what responses.create(...) gives back
.output — a list of items
[0] the message (the reply)
.content — a list of parts
[0].text → '{"company": "Apple", "eps": 1.72, …}'
[1], [2] …
other items, e.g.
reasoning or tool calls

`response.output[0].content[0].text` reads like opening a tiffin box: the box → its first compartment → the list inside → the first item → the text. **Shortcut:** `response.output_text` gives you the same text and is safer with reasoning models, where `output[0]` may be a reasoning item instead of the message.

1. `text={"format": {"type": "json_object"}}` switches on **JSON mode**: the model must return valid JSON. Always mention "JSON" in your prompt when you use it — the API expects that.
2. `json.loads(raw_text)` turns the JSON *text* into a Python *dictionary*, so you can write `summary["eps"]`.
3. `json.dumps(summary, indent=2)` goes the other way, only to print it nicely indented.

Same prompts, two other models

Python · Gemini 2.5 Flash (free tier)

```python
from google import genai

gclient = genai.Client(api_key=os.environ["GOOGLE_API_KEY"])   # separate name, so we don't overwrite 'cli
resp_genai = gclient.models.generate_content(
    model="gemini-2.5-flash",
    contents=system_prompt + "\n\n" + user_prompt,             # here: both prompts glued into one text
    config={"response_mime_type": "application/json"},         # Gemini's version of JSON mode
)
print(resp_genai.text)
```

Python · Llama 3.1 8B Instruct (open weights · needs a GPU, e.g. Colab T4)

```python
import torch, transformers

pipe = transformers.pipeline(
    "text-generation",
    model="meta-llama/Llama-3.1-8B-Instruct",       # downloaded to YOUR machine — nothing leaves it
    model_kwargs={"torch_dtype": torch.float16},     # half-precision numbers → fits on a 16 GB GPU
    device_map="auto",                               # place it on the GPU automatically
)
messages = [{"role": "system", "content": system_prompt},
            {"role": "user",   "content": user_prompt}]
outputs = pipe(messages, max_new_tokens=1024)
print(outputs[0]["generated_text"][-1]["content"])  # see the note below
```

1. `outputs[0]` — the pipeline returns one result per input; we sent one conversation, so take the first.
2. `["generated_text"]` — this holds the *whole chat*: your system message, your user message, and the model's new reply at the end.
3. `[-1]["content"]` — `-1` means "last item", i.e. the model's reply; `["content"]` is its text.

What came back — and the finance lesson hiding in it

KEY	GPT-4.1-MINI	GEMINI 2.5 FLASH	LLAMA 3.1 8B
`revenue`	`98700000000` a number	`"$98.7B (+6% YoY)"` text, with growth mixed in	`"$98.7 billion"` text
`eps`	`1.72`	`"$1.72 (+9% YoY)"`	`"$1.72"`
`guidance`	object with 3 fields	one sentence	object with 3 fields
`highlights`	Product-by-product detail: iPhone, Mac, iPad, Services record, margin, cash returned to shareholders	Key growth drivers grouped together	Short, high-level facts, little analysis

💰 **FINANCE CORNER — "VALID JSON" IS NOT THE SAME AS "USABLE JSON"**

All three replies are valid JSON, yet the fund manager's dashboard runs `summary["revenue"] > 90e9`. It works for GPT and **crashes** for the other two, because their revenue is text. In finance, numbers must arrive as numbers, in a known unit. Fix it in the prompt — name every key, its type and its unit:

Python · a stricter output contract

Copy

```
system_prompt = """You are a finance analyst writing for portfolio managers.
Return strict JSON with exactly these keys and types:
  company: string
  quarter: string, like "FY2025 Q3"
  revenue_usd: number in US dollars, no symbols or words (98700000000, not "$98.7B")
  revenue_growth_yoy_pct: number (6 means +6%)
  eps_usd: number
  guidance: string
  highlights: list of strings
  risks: list of strings
  outlook: one of "positive", "neutral", "negative"
"""
```

Even stricter: hand the API a full JSON Schema ("structured outputs"), so the model cannot return the wrong type at all. And notice the depth difference: the small 8B model listed facts, the bigger models explained *why* — size still matters for analysis.

1.5 Prompting, finance style

You know the techniques; here is what they look like on a trading desk.

TECHNIQUE	FINANCE EXAMPLE	USE WHEN
Zero-shot	"Classify the sentiment of this headline as positive, negative or neutral: *Apple reports record-breaking revenue in Q4 despite market slowdown*" → **Positive**	The task is common and simple
Few-shot	Show three labelled headlines first (see below), then ask for a new one	You want a house style or tricky labels
Chain-of-thought	Give a worked example of a profit-margin calculation, then a new company	The answer needs arithmetic or several steps
Zero-shot CoT	Add one sentence at the end: "Let's think step by step."	Multi-step, but you have no worked examples

```
Task: Classify the sentiment of a headline as Positive, Negative, or Neutral.
Headline: Tesla stock falls after earnings miss expectations → Negative
Headline: Amazon maintains steady growth amid economic slowdown → Neutral
Headline: Apple reports record-breaking revenue in Q4 despite market slowdown → Positive
Now classify: Nvidia shares surge after strong AI chip demand drives record quarterly revenue
Sentiment: ☐
```

One example = one-shot, n examples = n-shot.

💰 **FINANCE CORNER — NET PROFIT MARGIN**

Net profit margin = net income ÷ revenue. Out of every ₹100 of sales, how many rupees are profit? Profit going up is not enough — if sales went up even faster, the business is actually getting *less* efficient. The margin tells you which.

✏ **SIMULATION**

Run the chain-of-thought yourself

This is exactly the 3-step reasoning the CoT prompt teaches the model. Change the numbers (in $ millions) and watch each step.

Year 1 revenue	Year 1 net income	Year 2 revenue	Year 2 net income
2000	200	2400	260

Step 1 · Calculate both margins

Year 1: 200 ÷ 2000 = **10.00%** · Year 2: 260 ÷ 2400 = **10.83%**

Step 2 · Compare

10.00% → 10.83% (+0.83 percentage points)

Step 3 · Conclude

Improving The company keeps more of each dollar of sales.

Why: revenue grew 20.00%, net income grew 30.00%. The margin improves only when profit grows faster than sales.

⚠ **WATCH OUT — THE 100-EXAMPLE PROBLEM**

Few-shot is great with 3 examples. With 100 labelled headlines in every prompt, you pay for those tokens on every call, responses slow down, and you may exceed the context window. That is the moment to consider fine-tuning or retrieval instead.

1.6 Fine-tuning in finance — why it's rarely the first move

A fine-tuning dataset is just many rows of "here is the input, here is the answer I want". For headline sentiment, one training row looks like this:

JSONL · one training example per line

```
{"messages": [{"role": "system", "content": "You are a senior financial analyst."}, {"role": "user", "conte
```

Cheaper variants such as **PEFT / LoRA** train a small add-on instead of the whole model. Still, in finance fine-tuning hits four walls:

1. **Markets move weekly.** New regulations, rates and earnings make a fine-tuned model stale within weeks — and you must retrain to update it.
2. **Client data is confidential.** Uploading transaction records to a vendor's fine-tuning sandbox may break compliance rules.
3. **Decisions need live signals.** Credit risk or portfolio rebalancing depends on today's prices and macro data — far too fast to retrain for.
4. **Most bank data is messy.** PDFs, emails and memos must be hand-converted into clean input/answer pairs first.

WHAT YOU NEED	BEST FIT
A stable *style* or narrow task (e.g. label headlines our way)	Fine-tuning (or a small specialised model)
Fresh facts — prices, news, today's numbers	A tool that calls a data API
Answers from the bank's private documents	RAG over those documents
Several steps, decisions, checks	An agent (with tools + memory)

1.7 Lab 1B · Prove the knowledge cutoff with NVIDIA

A two-line experiment you can show in class. Ask a model with a known training cutoff (GPT-4.1: June 2024) for "the latest" news:

Python · lab_1b_knowledge_cutoff.py

```python
response = client.responses.create(
    model="gpt-4.1",
    input="Give me the latest news about NVIDIA.",
)
print(response.output_text)
```

```
Here are some of the latest news highlights about NVIDIA as of June 2024:
1. NVIDIA Surpasses $3 Trillion Market Cap …
2. NVIDIA Unveils Next-Gen Rubin GPUs …
```

> 💰 **FINANCE CORNER**
>
> A real event — but from the model's training window, served as "latest". Here this model was honest and stamped the date; others won't, and will confidently invent a "recent" headline. In finance, **stale is wrong**: a trader acting on last year's news loses money. Newer models have later cutoffs, but the gap never closes — so any "now" question must go through a tool.

1.8 Lab 1C · "What's NVIDIA trading at?" — without and with an agent

Without an agent

Python · lab_1c_price.py · plain LLM call

```python
response = client.responses.create(model="gpt-4.1", input="What is the last price of NVIDIA.")
print(response.output_text)
```

```
As of my latest update in June 2024, I do not have real-time data access. To find the most recent
price of NVIDIA (NVDA), please check Yahoo Finance, Google Finance or NASDAQ…
```

Polite, honest, useless to a trader. Now we give it a tool.

Step 1 — The tool: a normal Python function plus one decorator

Python · lab_1c_price.py · step 1 (pip install openai-agents yfinance)

```python
from agents import Agent, Runner, function_tool
import yfinance as yf

@function_tool                                    # turns this function into a tool the LLM can call
def get_latest_stock_price(symbol: str) -> dict:
    """Fetch the current stock price for the given symbol."""   # the LLM reads this to decide when to use
    ticker = yf.Ticker(symbol)                    # a handle for one stock, e.g. "NVDA"
    hist = ticker.history(period="1d")            # a table of the last trading day
    latest_close = hist["Close"].iloc[-1]         # column "Close", last row  → latest price
    latest_date = hist.index[-1]                  # label of that last row    → its date
    return {"price": float(latest_close), "date": str(latest_date.date())}
```

1. `@function_tool` is like printing a **menu card** for your function. The SDK reads the name, the docstring (the description) and the type hint `symbol: str` (the input), and shows that card to the LLM.

2. `yf.Ticker(symbol)` opens a handle to one stock. `.history(period="1d")` returns a pandas table (a DataFrame) for the last trading day — see the picture below.
3. `hist["Close"]` picks the Close column; `.iloc[-1]` means "row at position −1", i.e. the last row.
4. `hist.index[-1]` — the *index* is the row labels on the left (dates). The last label is the date of that price.
5. `float(...)` and `str(...date())` turn pandas types into plain Python values, so the result becomes clean text for the LLM.

INDEX (DATE)	OPEN	HIGH	LOW	CLOSE	VOLUME
2025-08-15	181.88	181.90	178.04	178.89	156,602,200

What `hist` looks like (illustrative numbers). Highlighted: `hist.index[-1]` on the left, `hist["Close"].iloc[-1]` on the right. With `period="5d"` you would get five rows, and `[-1]` would still pick the newest.

FINANCE CORNER — OHLCV

Every price table has the same five columns. **Open**: first trade of the day. **High / Low**: the day's extremes. **Close**: last trade — during market hours, the latest price so far. **Volume**: how many shares changed hands.

Step 2 — The agent, then run it

Python · lab_1c_price.py · step 2

Copy

```python
stock_price_agent = Agent(
    name="Stock Price Agent",                                          # a label, shows up in logs
    instructions="Fetch Historical prices for a given stock symbol",   # its job description
    model="gpt-4.1",                                                   # the brain
    tools=[get_latest_stock_price],                                    # what it may call
)

output = Runner.run_sync(                                              # run_sync: no 'await' needed
    starting_agent=stock_price_agent,
    input="What is today's date, and what is the current stock price of Nvidia?",
)
print(output.final_output)
```

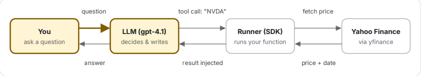
```
Today's date for the latest price data is August 15, 2025.
The current stock price of Nvidia (NVDA) is $178.89.
```

WHY ASK FOR THE DATE TOO?

It's a lie detector. If the date matches today's trading day, the answer came from the tool. If it shows a 2024 date, the model answered from memory. Also notice: you typed "Nvidia", but the tool needs a ticker — the LLM filled in `"NVDA"` itself. Choosing tool arguments is part of the LLM's job.

SIMULATION

Step through the agent loop



◀ Back | Next step ▶ | Step 1 of 6

You ask

`Runner.run_sync` sends your question to the LLM — together with the tool's "menu card": its name, its description (the docstring) and its input (`symbol: str`).

WHAT TRAVELS ON THE ARROW

```
input = "What is today's date, and what is the current stock price of Nvidia?"
```

WHERE BANKS TAKE THIS NEXT

- **Market intelligence agent** — watches news, economic indicators and central-bank statements, and summarises the likely impact on each sector.
- **Client reporting agent** — pulls each client's portfolio data, calculates performance and writes a quarterly report in that client's style.
- **Earnings-call analyser** — transcribes the call, extracts metrics, scores sentiment with evidence, flags guidance changes.
- **Backtesting agent** — you say "momentum strategy on tech stocks"; it fetches history, tests how that strategy *would have* performed, and charts the results.

SECTION 2

Agent design patterns — finance edition

You know the patterns. What's new here is **what they look like when money is on the line**, and the **code that builds each one** — six labs, each explained variable by variable.

> ⏩ **ALREADY COVERED — WE WON'T REPEAT**
>
> What tool use, working / episodic / semantic / procedural memory, RAG, guardrails, LLM-as-a-judge, sequential multi-agent orchestration and context management *are*. We use those words freely below and spend the time on finance knowledge and code.

PATTERN YOU KNOW	FINANCE LAB ON THIS PAGE
Tool use	2A · Pull Apple's ratios from a financial-data API
Working memory	2B · A rebalancing chat that remembers the portfolio
Episodic memory	2C · The rebalancing agent writes a "journal" and reads it next time
Planning, ReAct, self-consistency	2.5 · Trend detection, live price with ReAct (Lab 2D), earnings-surprise vote
LLM-as-a-judge (evaluator–optimizer)	2E · A news agent whose work is checked by a second agent
Sequential orchestration	2F · Retrieval → Analysis → Buy/Hold/Sell

2.1 Finance crash course: the three ratio families

Labs 2A and 2F feed financial ratios to agents. If you can't read the ratios, you can't tell whether the agent's conclusion is sensible — so let's learn them first.

> 🧠 **ANALOGY — YOUR FRIEND WANTS YOU TO INVEST IN HER CAFÉ**
>
> - **Profitability** — "Does the café actually keep money from each coffee it sells?"
> - **Valuation** — "Is the price she's asking for a share of the café fair, compared with what it earns?"
> - **Liquidity** — "Can it pay this month's milk supplier and rent from the cash in the till and bank?"

RATIO	FORMULA (IN WORDS)	READ IT AS
Profitability — how efficiently it makes money		
Gross margin	(Revenue − cost of making the products) ÷ revenue	Of every $100 sold, what's left after production cost
Operating margin	Operating income ÷ revenue	...after also paying salaries, R&D, marketing
Net profit margin	Net income ÷ revenue	...after interest and tax too — the true profit per $100
Return on assets (ROA)	Net income ÷ total assets	Profit per $1 of everything the company owns
Return on equity (ROE)	Net income ÷ shareholders' equity	Profit per $1 of the owners' money on the books
Valuation — is the price tag reasonable?		
Price-to-earnings (P/E)	Share price ÷ EPS	Dollars you pay for $1 of yearly profit
Price-to-book (P/B)	Market value ÷ shareholders' equity	Price vs the accounting value of the company
Price-to-cash-flow / free-cash-flow	Market value ÷ operating (or free) cash flow	Like P/E, but using real cash instead of accounting profit
EV-to-EBITDA	(Market value + debt − cash) ÷ earnings before interest, tax, depreciation & amortisation	Price of the *whole* business vs its operating earnings
EV-to-sales	Enterprise value ÷ revenue	How many years of sales the whole business costs
Liquidity — can it pay the bills due this year?		
Current ratio	Current assets ÷ current liabilities	Short-term resources per $1 of bills due within a year
Quick ratio	(Current assets − inventory) ÷ current liabilities	Same, counting only things that turn into cash fast
Cash ratio	Cash & equivalents ÷ current liabilities	The strictest test: cash alone vs the bills
Working capital	Current assets − current liabilities	An amount, not a ratio: the short-term cushion

Apple through the ratio lens (values our agents fetched for fiscal 2025)

Profitability strong

Gross margin
46.9%
~$47 of every $100 sold is left after making the products.

Operating margin
32.0%
$32 left after running the whole company.

Net profit margin
26.9%
~$27 true profit per $100 of sales.

Return on assets
30.9%
31 cents of profit a year per $1 it owns.

Return on equity
171%
Looks impossible. It isn't — see the simulation below.

Valuation expensive

P/E
36.4
Investors pay ~$36 for each $1 of yearly profit.

P/B
55.3
Price is 55× book value — same small-equity effect as ROE.

EV / EBITDA
28.7
The whole business costs ~29 years of operating earnings.

EV / Sales
10.0
~10 years of revenue.

Liquidity below 1 — but read on

Current ratio
0.89
$0.89 of short-term assets per $1 of bills due this year.

Quick ratio
0.57
Same, without inventory.

Cash ratio
0.33
Cash alone covers 33 cents per $1.

SIMULATION

Why is Apple's ROE 171%? Shrink the denominator.

Profit stays fixed at **$100**. Now the company uses its cash to **buy back its own shares**, which shrinks shareholders' equity on the balance sheet. Drag the slider:

Shareholders' equity: $500

ROE = profit ÷ equity = $100 ÷ $500 = **20%**
A normal-looking ROE.
Price-to-book has the same denominator (equity), which is why Apple's P/B is 55.

⚠ THE LESSON FOR AGENT BUILDERS

A ratio is a fraction — always look at the denominator. Same with liquidity: in our run the agent said Apple's current ratio below 1 signals "potential liquidity issues". A human analyst would add that Apple generates very large operating cash flow every quarter and keeps big long-term investments, so a ratio under 1 is a deliberate choice, not distress. **The tool makes the numbers trustworthy; the interpretation is still the LLM's** — and it can be shallow. Ask for comparisons with peers and with the company's own history in your prompt.

2.2 Lab 2A · The ratio retriever (tool use)

The request: "Give me Apple's liquidity ratios for the latest fiscal year, and explain what each one means." We'll use **Finance Toolkit**, a Python library that pulls financial statements from the Financial Modeling Prep API and calculates dozens of ratios for you.

Step 1 — Imports and keys

Python · lab_2a_ratio_tool.py · step 1 (pip install openai-agents financetoolkit) Copy

```python
import os
import pandas as pd
from financetoolkit import Toolkit
from agents import Agent, Runner, function_tool

OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]
FINANCIAL_MODELING_PREP_API_KEY = os.environ["FINANCIAL_MODELING_PREP_API_KEY"]
```

Step 2 — The tool: fetch everything, keep only what we need

Python · lab_2a_ratio_tool.py · step 2 Copy

```python
@function_tool
def get_company_fundamentals(ticker: str) -> dict[str, pd.DataFrame]:
    """Get profitability, valuation and liquidity ratios for a given ticker."""
    companies = Toolkit(
        [ticker], api_key=FINANCIAL_MODELING_PREP_API_KEY, start_date="2022-01-01"
    )
    ratios = companies.ratios.collect_all_ratios()     # ONE big table: rows = ratio names, columns = year

    # Our three "shopping lists"
    profitability_ratios_names = ["Return on Equity", "Return on Assets", "Gross Margin",
                                  "Operating Margin", "Net Profit Margin"]
    valuation_ratios_names = ["Price-to-Earnings", "Price-to-Book", "Price-to-Cash-Flow",
                              "Price-to-Free-Cash-Flow", "EV-to-EBITDA", "EV-to-Sales"]
    liquidity_ratios_names = ["Current Ratio", "Quick Ratio", "Cash Ratio", "Working Capital",
                              "Operating Cash Flow Ratio", "Operating Cash Flow to Sales Ratio",
                              "Short Term Coverage Ratio"]

    return {
        "profitability_ratios": ratios.loc[ratios.index.intersection(profitability_ratios_names)],
        "valuation_ratios":     ratios.loc[ratios.index.intersection(valuation_ratios_names)],
        "liquidity_ratios":     ratios.loc[ratios.index.intersection(liquidity_ratios_names)],
    }
```

① `Toolkit([ticker], api_key=..., start_date="2022-01-01")` — "open an account for this list of tickers, from 2022 onwards". It takes a *list* even for one company, because it can compare many.

② `collect_all_ratios()` returns one big DataFrame with dozens of rows (one per ratio) and one column per year.

③ The three `..._names` lists are our **shopping lists**: the rows we actually want.

④ `ratios.index.intersection(names)` — "which of my shopping-list names really exist as row labels in the table?" Names that don't exist are quietly dropped.

⑤ `ratios.loc[...]` then picks exactly those rows. Without the intersection, one misspelt name would crash `.loc` with a `KeyError`.

⑥ The tool returns a dictionary of three small tables. The SDK turns this into text for the LLM.

SIMULATION

Watch `.index.intersection()` + `.loc[]` pick rows

Profitability list | **Valuation list** | **Liquidity list** | **List with a typo**

Your shopping list (green = found in the table, red = not found → dropped):

`Return on Equity` `Return on Assets` `Gross Margin` `Operating Margin` `Net Profit Margin`

INDEX (RATIO NAME)	2025
`Gross Margin`	0.4691
`Operating Margin`	0.3197
`Net Profit Margin`	0.2692
`Return on Assets`	0.3093
`Return on Equity`	1.7142
`Debt-to-Equity Ratio`	…
`Interest Coverage Ratio`	…
`Price-to-Earnings`	36.418
`Price-to-Book`	55.3236

`ratios.index.intersection(list)` → 5 of 5 names found → `ratios.loc[...]` returns those **5 highlighted rows**.

PRO TIP

The SDK converts whatever your tool returns into text. Small DataFrames print fine; for larger ones, return `df.round(3).to_dict()` so the LLM gets compact, clean numbers instead of a wide printed table.

Step 3 — The agent and the question

Python · lab_2a_ratio_tool.py · step 3

Copy

```python
fundamental_retrieval_agent = Agent(
    name="Fundamental Retrieval Agent",
    model="gpt-4o-mini",
    instructions="""
You are a Fundamental Retrieval Agent.
Tasks:
1) Call the fundamentals tool for the specified ticker.
2) Output the desired set of ratios.
""",
    tools=[get_company_fundamentals],
)

fundamental_results = await Runner.run(fundamental_retrieval_agent, """Proceed step by step:
1- List 3 liquidity ratios and explain their significance.
2- Provide Apple's values for the three listed liquidity ratios for the most
recent fiscal year, knowing that we are in 2026.
""")
print(fundamental_results.final_output)
```

① The question is written as numbered steps — first explain, then fetch. Step-by-step requests give tidier, more complete answers.

② `knowing that we are in 2026` tells the model today's year, so it picks the right "most recent fiscal year" instead of guessing from its training cutoff. Small line, big difference.

```
### Apple's Liquidity Ratios for the Most Recent Fiscal Year (2025)
1. Current Ratio: 0.8933 — a ratio below 1 suggests potential liquidity issues.
2. Quick Ratio: 0.5704 — …
3. Cash Ratio: 0.3302 — …
```

The figures are real (from the API). The comment "potential liquidity issues" is the LLM's own reading — exactly the shallow interpretation the warning above told you to question.

2.3 Lab 2B · Working memory: a rebalancing chat that remembers

FINANCE CORNER — REBALANCING IN 5 WORDS

- **Asset allocation** — how the money is split: e.g. 70% stocks, 20% bonds, 10% cash.
- **Rebalancing** — buying and selling to move that split back to where it should be.
- **Volatility regime** — is the market calm (low) or jumpy (high) right now?
- **Market trend** — rising (bullish) or falling (declining / bearish)?
- **Defensive sectors** — Healthcare and Utilities: people still buy medicine and electricity in a recession, so these hold up better.

A typical rule set: *no sector above 35%* (avoid putting too many eggs in one basket) and *at least 30% in defensive sectors.*

SIMULATION

Be the rebalancing agent: check the rules, apply the trades

Edit any weight (%) or rule. The checker does what the agent's "reasoning steps" do.

Tech (%)	Healthcare (%)	Financials (%)	Energy (%)	Industrials (%)
40	25	20	10	5

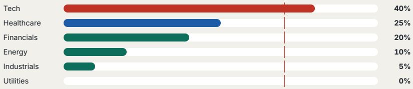
Utilities (%)

0

Rule: max per sector (%)	Rule: min Healthcare + Utilities (%)
35	30

Apply agent's proposal | Apply client's tweak | Reset

Agent's proposal: sell 8% Tech, buy 6% Healthcare + 2% Utilities. Client's tweak: exit Energy completely, put that 10% into Utilities ("I want stable dividends").



Red tick = sector cap. Blue = defensive sectors.

✗ **Sector cap broken:** Tech 40% > 35%

✗ **Defensive too low:** Healthcare + Utilities = 25% < 30%

✓ **Weights add up to 100%**

Step 1 — The agent

Python · lab_2b_working_memory.py · step 1 Copy

```python
from agents import Agent, Runner, SQLiteSession

portfolio_agent = Agent(
    name="Portfolio Rebalancing Agent",
    model="gpt-4o-mini",
    instructions="""You are a Portfolio Rebalancing Agent for a professional investor.
Your goal is to propose clear, explainable rebalancing actions that keep the
portfolio aligned with the target risk/return profile and constraints.
Your role
- Analyze the current portfolio (weights, asset classes, sectors, factors, cash).
- Incorporate market regime information (volatility, trends, macro environment).
- Compare current allocation to:
  - Target allocation or risk budget (if provided),
  - Or to a reasonably diversified allocation based on the information you have.
- Propose specific rebalancing trades with approximate percentage changes and short rationales.
- Prioritize diversification, risk control, and alignment with the investor's objectives.
- Keep answers concise and structured, and state assumptions when information is missing.
- Treat the allocation provided in the user's prompt as the portfolio's current allocation.
Always explain your reasoning.""",
)
```

Two lines matter most in finance: **"state assumptions when information is missing"** (an advisor must say "I assumed you want capital protection" rather than silently guessing) and **"always explain your reasoning"** (every recommendation must be explainable to a client or a regulator).

Step 2 — Create the notepad (the session)

Python · lab_2b_working_memory.py · step 2 Copy

```python
session_ptf_rebal = SQLiteSession("conversation_ptf")    # a fresh notepad named "conversation_ptf"

# Same, but saved in a file so it survives a restart:
# session_ptf_rebal = SQLiteSession("conversation_ptf", "portfolio_sessions.db")
```

ANALOGY — THE TRADER'S DESK NOTEPAD

All day a trader scribbles open positions and the client's instructions on a notepad. At 4 pm the notepad goes in the bin. `SQLiteSession` is that notepad: by default it lives only in your computer's RAM and vanishes when the program stops. The name `"conversation_ptf"` is the label on the notepad, so you could keep several clients' notepads apart.

Step 3 — Two turns, same notepad

Python · lab_2b_working_memory.py · step 3

Copy

```python
input_user_1 = """Current Market Situation:
- Volatility Regime: high
- Market Trend: declining
Current Portfolio State:
- Portfolio: {"stocks": 0.70, "bonds": 0.20, "cash": 0.10}
What rebalancing action should I take?"""

output_1 = await Runner.run(
    starting_agent=portfolio_agent,
    input=input_user_1,
    session=session_ptf_rebal,      # read what's on the notepad, then write this turn onto it
)
print(output_1.final_output)        # suggests ~50% stocks / 40% bonds / 10% cash (defensive)

input_user_2 = "In this case, I'm reducing equity to 50%, increasing bonds to 35% and cash to 15%."
output_2 = await Runner.run(
    starting_agent=portfolio_agent,
    input=input_user_2,
    session=session_ptf_rebal,      # SAME notepad → the agent still knows the 70/20/10 starting point
)
print(output_2.final_output)
```

Notice `input_user_1` is a plain string, so its curly braces are just characters. Remember this — in Lab 2C, braces become special.

SIMULATION

What does the LLM actually receive on turn 2?

With session **Without session**

USER · TURN 1
Volatility high, trend declining. Portfolio: stocks 0.70, bonds 0.20, cash 0.10. What rebalancing action should I take?

ASSISTANT · TURN 1
Assumption: capital preservation. Suggest stocks 50%, bonds 40%, cash 10% ...

USER · TURN 2
In this case, I'm reducing equity to 50%, increasing bonds to 35% and cash to 15%.

With the session, the model sees all three messages. It knows you started at 70 / 20 / 10, so it can size the trades: **sell 20% stocks, buy 15% bonds, add 5% cash**.

Step 4 — Read the notepad back (the part that confuses everyone)

Before we write any code, look at **what the session stores**. User messages and assistant messages are stored in *different shapes*:

```
all_items = [
  { "role": "user",
    "content": "Current Market Situation:  ..." },           ← a plain string
  { "role": "assistant",
    "content": [ { "type": "output_text",
                   "text": "### Current Portfolio..." } ] },  ← a LIST of parts
  { "role": "user",
    "content": "In this case, I'm reducing..." },
  { "role": "assistant",
    "content": [ { "type": "output_text", "text": "..." } ] },
]
```

ANALOGY — LETTER VS ENVELOPE

A user message is a **single letter**: read it directly. An assistant message is an **envelope** that can hold several pages (text, citations, ...). To read it you open the envelope and take page 0's text: `content[0]["text"]`.

Python · lab_2b_working_memory.py · step 4 (simplified, same result)

Copy

```python
all_items = await session_ptf_rebal.get_items()    # everything on the notepad, oldest first
conversation_history = []                           # we'll fill this with (role, text) pairs

for i, msg in enumerate(all_items, 1):              # i counts 1, 2, 3 … (only for printing)
    role = msg.get("role", "unknown")               # "user" or "assistant"; tool-call items have no role
    content = msg.get("content", "")

    if role == "user":
        text = content                              # a letter: already a string
    elif role == "assistant":
        text = content[0]["text"]                   # an envelope: open it, take page 0's text
    else:
        continue                                    # skip tool calls and other item types

    print(f"{i}. {role}: {text}")
    print("--" * 50)
    conversation_history.append((role, text))       # save a clean (role, text) pair
```

1. `.get("role", "unknown")` is a safe lookup: if an item has no `"role"` key (tool calls don't), you get `"unknown"` instead of a crash.

2. `enumerate(all_items, 1)` gives you a counter `i` starting at 1 alongside each item — just for numbered printing.

3. `conversation_history` ends up as a simple list of pairs: `[("user", "Current Market…"), ("assistant", "### Current Portfolio…"), …]`. **This variable is the bridge into Lab 2C.**

WHY THIS MATTERS ON A TRADING DESK

Without the session, turn 2 ("reduce equity to 50%") arrives with no starting point — the agent can't know you came from 70/20/10, can't size the trades, and might invent a starting portfolio. For a bank, "the assistant forgot what the client said two minutes ago" is not a UX bug; it is a wrong trade.

2.4 Lab 2C · Episodic memory: from today's chat to tomorrow's wisdom

In Lab 2B the notepad is binned when the session ends. Now we teach the agent to **write a journal entry before the notepad is binned**, file it in a cabinet that survives restarts, and **read the most relevant entries** the next time a client asks a similar question.

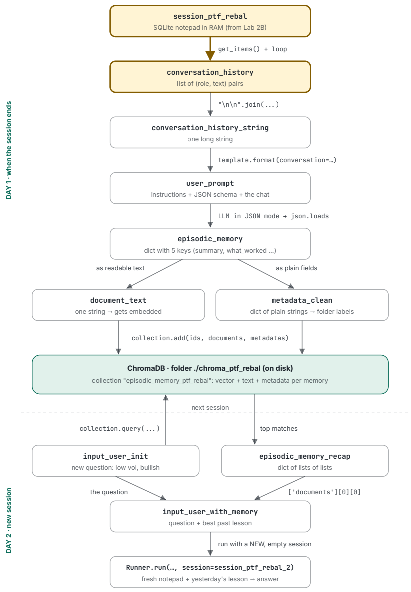
ANALOGY — THE TRADER'S END-OF-DAY JOURNAL

At 4 pm, before throwing away the notepad, a good trader writes one journal page: *what the market was doing, what we changed, what worked, what didn't, lessons.* The page is filed in a cabinet sorted by **meaning**, not by date. Next month, facing a similar market, she pulls the three most similar pages before deciding. That is the whole lab.

MEMORY TYPE	IN A BANK, IT HOLDS...
Working	Today's notepad: current allocation, the client's constraints, this session's reasoning
Episodic	The journal: past rebalancing sessions, what was proposed, what the client accepted or rejected
Semantic	The rulebook and facts: "max 75% in stocks", "Apple is a technology company", "USD/EUR is a currency pair", allowed order types
Procedural	The SOP checklist: fetch ratios → check news sentiment → check sector exposure → compare with targets → write recommendation with risk notes. Can come from the bank's documented processes (KYC review, underwriting steps) or be learned — *with controls, so the agent doesn't learn a bad habit.*

The big picture: how your data travels

This is the map for the whole lab. Every box is a **variable** in the code; every arrow is **the line of code** that turns one into the next. Use the step-through below it — the map lights up as you go.


DAY 1 · when the session ends
session_ptf_rebal
SQLite notepad in RAM (from Lab 2B)
get_items() + loop
conversation_history
list of (role, text) pairs
"\n\n".join(...)
conversation_history_string
one long string
template.format(conversation=…)
user_prompt
instructions + JSON schema + the chat
LLM in JSON mode → json.loads
episodic_memory
dict with 5 keys (summary, what_worked ...)
as readable text
as plain fields
document_text
one string → gets embedded
metadata_clean
dict of plain strings → folder labels
collection.add(ids, documents, metadatas)
ChromaDB · folder ./chroma_ptf_rebal (on disk)
collection "episodic_memory_ptf_rebal": vector + text + metadata per memory
next session
collection.query(...)
top matches
DAY 2 · new session
input_user_init
new question: low vol, bullish
episodic_memory_recap
dict of lists of lists
the question
['documents'][0][0]
input_user_with_memory
question + best past lesson
run with a NEW, empty session
Runner.run(…, session=session_ptf_rebal_2)
fresh notepad + yesterday's lesson → answer

Day 1 turns a chat into a journal page and files it on disk. Day 2 starts with an empty notepad, searches the cabinet by meaning, and pastes the best page into the new question. **Long-term memory = retrieve, then paste into the prompt.**

SIMULATION · THE KEY ONE

Step through the lab and watch every variable fill up

The trail below is a mini version of the map: **gold** = this step, **green** = already filled.

◀ Back | Next step ▶ | Step 1 of 9

DAY 1 `session_ptf_rebal` → `conversation_history` → `conversation_history_string` → `user_prompt` → `episodic_memory` → `document_text` + `metadata_clean` → `Chroma collection` **DAY 2** `input_user_init` → `episodic_memory_recap` → `input_user_with_memory` → `Runner.run (new session)`

1 · Read the notepad

The session lives in RAM and dies with the notebook. First we copy it into a clean Python list of (who, what) pairs.

CODE

```
all_items = await session_ptf_rebal.get_items()
# ...the loop from Lab 2B builds:
conversation_history.append((role, text))
```

VARIABLE NOW HOLDS · `CONVERSATION_HISTORY`

```
[
  ("user", "Current Market Situation: volatility high, trend declining. Portfolio {stocks 0.70,
bonds 0.20, cash 0.10} ..."),
  ("assistant", "### Current Portfolio Analysis ... target: stocks 50%, bonds 40%, cash 10% ..."),
  ("user", "In this case, I'm reducing equity to 50%, increasing bonds to 35% and cash to 15%."),
  ("assistant", "### Summary of Final Rebalanced Portfolio: stocks 50%, bonds 35%, cash 15% ...")
]
```

Now the code, one step at a time

Step 1 · Start from Lab 2B's output. We continue the same notebook, so `conversation_history` already exists — the list of `(role, text)` pairs we built from the notepad.

Step 2 · Flatten the chat into one string. An LLM prompt is one piece of text, not a Python list.

Python · lab_2c_episodic_memory.py · step 2 — Copy

```python
# Original one-liner:
conversation_history_string = "\n\n".join([f'{elem[0]}: """{elem[1]}"""' for elem in conversation_history]

# Exactly the same thing, with friendlier names:
conversation_history_string = "\n\n".join(
    [f'{role}: """{text}"""' for role, text in conversation_history]
)
```

① `elem` is one pair like `("user", "Current Market…")`, so `elem[0]` is the role and `elem[1]` is the text. Unpacking it as `role, text` is easier to read.

② The `[ … for … in … ]` part (a list comprehension) builds one small string per message: `user: """Current Market…"""`. The triple quotes fence each message so the LLM sees exactly where one speaker stops.

③ `"\n\n".join(list)` glues those small strings into one, with a blank line between each.

```
# BEFORE: a list of pairs
[("user", "Current Market Situation: ... What rebalancing action should I take?"),
 ("assistant", "### Current Portfolio Analysis ..."), ...]

# AFTER: one string
user: """Current Market Situation: ... What rebalancing action should I take?"""

assistant: """### Current Portfolio Analysis ..."""

user: """In this case, I'm reducing equity to 50% ..."""
```

Step 3 · The prompt template (shortened). It tells the LLM what a good journal page looks like.

Python · lab_2c_episodic_memory.py · step 3 — Copy

```python
episodic_memory_prompt_template = """
You will read a conversation between a user and a portfolio-rebalancing agent.
Extract an episodic memory that will help the agent in future sessions.

OUTPUT FORMAT:
Return only valid JSON in this exact schema:
{{
  "conversation_summary": string,            // 3-5 sentences: market conditions, portfolio situation, acti
  "portfolio_allocation_trajectory": string, // how allocations changed, with the market situation each ti
  "what_worked": string,                     // alignment with goals, risk management, clear decisions
  "what_didnt_work": string,                 // unclear strategy, over/under-reaction, concentration risk,
  "key_points": ["string"]                   // the most important insights to keep for future sessions
}}

(… a few examples of GOOD and POOR summaries go here — few-shot …)

Here is the conversation between the user and the principal portfolio-rebalancing agent:
{conversation}
"""

user_prompt = episodic_memory_prompt_template.format(conversation=conversation_history_string)
```

⚠ THE CURLY-BRACE TRAP — READ THIS TWICE

`.format()` treats every `{something}` as a slot to fill. That's what we want for `{conversation}`. But the JSON example also has braces! Writing them **doubled** — `{{` and `}}` — tells `.format()` "this is a real brace, leave it alone". After formatting, `{{` becomes `{`.

```
# In the template                       # After .format(conversation=...)
{{                                        {
  "conversation_summary": string,           "conversation_summary": string,
}}                                        }
{conversation}                            user: """Current Market Situation: ..."""  (the whole chat)
```

💰 FINANCE CORNER — "PORTFOLIO_ALLOCATION_TRAJECTORY"

This is the most finance-specific field. It records *how the allocation moved and what the market was doing each time*, e.g. `[{market: {volatility: high, trend: declining}, allocation: {stocks: 0.70, bonds: 0.20, cash: 0.10}}, …]`. Next time the market looks similar, the agent can see what this client did then — that's case-based reasoning, the way experienced advisors think.

Step 4 · Ask the LLM to write the journal page.

Python · lab_2c_episodic_memory.py · step 4 Copy

```python
import json
from openai import OpenAI

client = OpenAI(api_key=OPENAI_API_KEY)
response = client.responses.create(
    model="gpt-4.1-mini",
    input=[{"role": "user", "content": user_prompt}],
    text={"format": {"type": "json_object"}},              # JSON mode, as in Lab 1A
)
episodic_memory = json.loads(response.output[0].content[0].text)   # JSON text → Python dict
print(json.dumps(episodic_memory, indent=2))
```

```
episodic_memory = {
  "conversation_summary": "Initially, with high volatility and a declining market, the user reduced equity
  "portfolio_allocation_trajectory": "[{market: {volatility: high, trend: declining}, allocation: {stocks:
  "what_worked": "Progressive adjustments aligned well with market conditions ...",
  "what_didnt_work": "Sizable shifts without discussing the user's risk tolerance ...",
  "key_points": [ "High volatility + declining market justified cutting equity",
                  "Rebalancing should consider risk profile and trading costs", ... ]   ← a LIST
}
```

Step 5 · Prepare two versions of the same page.

Python · lab_2c_episodic_memory.py · step 5 Copy

```python
# Version A: one readable text — this is what gets embedded and searched by meaning
document_text = f"""
Conversation Summary: {episodic_memory['conversation_summary']}
Portfolio Allocation Trajectory: {episodic_memory['portfolio_allocation_trajectory']}
What Worked: {episodic_memory['what_worked']}
What Didn't Work: {episodic_memory['what_didnt_work']}
Key Points: {', '.join(episodic_memory['key_points'])}
"""

# Version B: the same facts as separate fields — the labels on the folder
metadata_clean = {
    "conversation_summary": episodic_memory["conversation_summary"],
    "portfolio_allocation_trajectory": episodic_memory["portfolio_allocation_trajectory"],
    "what_worked": episodic_memory["what_worked"],
    "what_didnt_work": episodic_memory["what_didnt_work"],
    "key_points": " | ".join(episodic_memory["key_points"]),   # list → one string
}
```

① `document_text` is the **journal page**. Chroma will turn it into a vector so it can be found by meaning.

② `metadata_clean` holds the **labels on the folder**: the same facts as named fields, useful for filtering and for reading one field back without re-parsing the text.

③ `" | ".join(...)` — Chroma metadata is meant for simple values (text, numbers, true/false), so the `key_points` list is joined into one string like `"High vol… | Rebalancing should…"`.

Step 6 · Open the filing cabinet.

Python · lab_2c_episodic_memory.py · step 6 (pip install chromadb) Copy

```python
import chromadb
from chromadb.utils import embedding_functions

# The translator: turns any text into a list of 1,536 numbers (a vector) that captures its meaning
embedding_function = embedding_functions.OpenAIEmbeddingFunction(
    api_key=OPENAI_API_KEY,
    model_name="text-embedding-3-small",
)

collection_name = "episodic_memory_ptf_rebal"                        # the drawer's name
persistent_client = chromadb.PersistentClient(path="./chroma_ptf_rebal")  # the cabinet, saved in a folder
collection = persistent_client.get_or_create_collection(              # open the drawer (create it if new,
    collection_name, embedding_function=embedding_function
)
```

🧠 **ANALOGY — CABINET, DRAWER, TRANSLATOR**

`PersistentClient(path=…)` is the **filing cabinet**, stored in the folder `./chroma_ptf_rebal` on disk — unlike the RAM notepad, it survives restarts. A **collection** is one drawer in it. The **embedding function** is a translator that writes every page's meaning as numbers, so "jumpy falling market" and "high volatility, declining trend" end up close together. `get_or_create` means re-running the cell won't fail.

Step 7 · File the page.

Python · lab_2c_episodic_memory.py · step 7 Copy

```
collection.add(
    ids=["memory_1"],               # a unique name for this page
    documents=[document_text],      # the text → Chroma embeds it automatically with our translator
    metadatas=[metadata_clean],     # the folder labels
)
```

Why lists? Because `add` can file many pages at once. Position 0 of each list belongs together: `ids[0]` ↔ `documents[0]` ↔ `metadatas[0]`.

⚠️ **SAME ID TWICE?**

Re-running `add` with `"memory_1"` again won't create a second page — Chroma skips the existing id (with a warning). For a real journal, give every session a unique id, e.g. `f"memory_{datetime.now():%Y%m%d_%H%M%S}"`, or use `collection.upsert(...)` to overwrite on purpose.

Step 8 · Days later: a new question searches the cabinet.

Python · lab_2c_episodic_memory.py · step 8 Copy

```
input_user_init = """Current Market Situation:
- Volatility Regime: low
- Market Trend: bullish
What rebalancing action should I take?"""

episodic_memory_recap = collection.query(
    query_texts=[input_user_init],   # a LIST: you could ask several questions at once
    n_results=3,                     # up to 3 closest pages per question
)
```

Chroma translates the question into a vector with the same translator, then returns the pages whose vectors are closest. The result's shape is where most people get lost:

```
episodic_memory_recap = {
  "ids":       [ [ "memory_1", "memory_7", "memory_3" ] ],
  "documents": [ [ "Conversation Summary: ...", "Conversation Summary: ...", "..." ] ],
  "distances": [ [ 0.21, 0.35, 0.48 ] ],          ← smaller = more similar
  "metadatas": [ [ {...}, {...}, {...} ] ],
}       outer list = one entry per question  ·  inner list = matches, best first
```

🧪 **SIMULATION**

Decode `['documents'][q][r]` — click any cell

Imagine you passed **two** questions in `query_texts`. Each row is one question; each column is a match, best first.

	[Q][0] BEST	[Q][1]	[Q][2]
q=0 · "low vol, bullish — what now?"	`memory_1 · 0.21`	`memory_7 · 0.35`	`memory_3 · 0.48`
q=1 · "client wants more dividends"	`memory_4 · 0.18`	`memory_1 · 0.4`	`memory_9 · 0.52`

`episodic_memory_recap['documents'][0][0]`
= the **best** matching page for question **0** ("low vol, bullish — what now?")
Its id: `episodic_memory_recap['ids'][0][0]` → `'memory_1'` · distance 0.21
This is the cell Lab 2C uses — first question, best match.

Step 9 · Paste the lesson into the new question and run with a fresh notepad.

Python · lab_2c_episodic_memory.py · step 9 Copy

```
session_ptf_rebal_2 = SQLiteSession("conversation_ptf_2")      # a brand-new, EMPTY notepad

input_user_with_memory = input_user_init + f"""
Here are relevant insights from past rebalancing episodes. Use them as guidance, not strict rules:
{episodic_memory_recap['documents'][0][0]}
Argue your proposition using past rebalancing episodes if needed.
"""

output_using_memory = await Runner.run(
    starting_agent=portfolio_agent,
    input=input_user_with_memory,
    session=session_ptf_rebal_2,
)
print(output_using_memory.final_output)
```

1. `episodic_memory_recap['documents'][0][0]` = "for my first (only) question, the best-matching journal page".
2. The `+` glues the old question and the memory block into one prompt: `input_user_with_memory`.
3. The session is **new and empty** — yet the agent knows yesterday's lesson, because we pasted it into the prompt. That is the entire trick of long-term memory.

WHY "GUIDANCE, NOT STRICT RULES"?

Last time the market was *high-volatility and falling*; today it's *low-volatility and rising*. Copying last time's defensive moves would be wrong. The phrase tells the agent to learn from the pattern (e.g. "this client cares about risk control") without repeating the trade. Good advisors do exactly this.

MAKING IT PRODUCTION-READY

- Use all top matches, not just one: `"\n---\n".join(episodic_memory_recap["documents"][0])`.
- With only one page in the drawer, `n_results=3` simply returns that one page.
- Write the journal page automatically when a session closes, with a unique id.

Variable cheat-sheet for Lab 2C

VARIABLE	TYPE	LIVES IN	WHAT IT HOLDS
`session_ptf_rebal`	SQLiteSession	RAM	Today's notepad (gone on restart)
`conversation_history`	list of (role, text)	RAM	Clean copy of the chat
`conversation_history_string`	str	RAM	The chat as one text
`episodic_memory_prompt_template`	str with a `{conversation}` slot	RAM	Instructions for writing a journal page
`user_prompt`	str	RAM	Template with the chat filled in
`episodic_memory`	dict (5 keys)	RAM	The journal page as data
`document_text`	str	RAM → disk	The page as readable text (gets embedded)
`metadata_clean`	dict of str	RAM → disk	Folder labels
`collection`	Chroma collection	Disk `./chroma_ptf_rebal`	The drawer of all journal pages
`input_user_init`	str	RAM	The new day's question
`episodic_memory_recap`	dict of lists of lists	RAM	Search results, best first
`input_user_with_memory`	str	RAM	New question + best past page
`session_ptf_rebal_2`	SQLiteSession	RAM	The new, empty notepad

2.5 Reasoning patterns on a trading desk

The planning variants, mapped to jobs an analyst at Future with Shivank Bank actually does:

PATTERN	IN ONE LINE	FINANCE EXAMPLE
Plan-then-execute	Write the full plan, then follow it	Rebalance: fetch allocations → find risk gaps → build the trade list → execute in order (re-plan only if a trade fails)
Plan-and-execute	Plan a little, act, look, re-plan	Company check: fetch ratios + news → spot a surprise jump in debt → *add* a cash-flow analysis step → continue
Hierarchical	Big goal → sub-plans → concrete actions	"Cut portfolio volatility" → rebalance equities / shorten bond duration / hedge currency → specific trades and % changes
Planning + ReAct	Each plan step runs its own think → act → observe loop	Invest or not: statements → ratios (falling margins? fetch more) → news sentiment → verdict
Planning + reflection	Critique the plan before running it	The review notices "no cash-flow quality check" and adds one before any data is fetched

FOUR WORDS FROM THAT TABLE

- **Leverage** — how much the company runs on borrowed money. A sudden jump is a red flag.
- **Bond duration** — how sensitive a bond's price is to interest-rate changes. Shorter duration = less swing.
- **Currency hedge** — a contract that protects you if, say, the dollar falls against the rupee.
- **Cash-flow quality** — are reported profits backed by real cash coming in? Profits without cash are a warning sign.

Worked example — a plan-then-execute trend detector for Apple

The plan an LLM would write: **(1)** get the last 3 months of daily prices · **(2)** compute the 20-day moving average · **(3)** compare today's average with the one a week ago · **(4)** label it Uptrend or Downtrend.

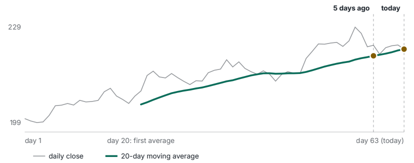
ANALOGY — A BATTER'S 20-INNINGS AVERAGE

One innings can be a fluke. A batter's average over the last 20 innings tells you their real form — and whether that average is rising or falling tells you the trend. A **20-day moving average** does the same for a stock: the average of the last 20 closing prices, recalculated every day.

SIMULATION

Run the plan on a made-up stock

Rising market | Falling market | Random market



Step 3: MA20 today = 222.13, five trading days ago = 220.01 → **Step 4:** Uptrend

The green line only starts on day 20 — you need 20 prices before the first average exists (that's why `ma20` is empty/NaN for the first 19 rows). Notice how much smoother it is than the grey daily line.

Python · trend_check.py · the same plan, as plain code

```python
import yfinance as yf

prices = yf.Ticker("AAPL").history(period="3mo")["Close"]      # step 1: ~63 daily closing prices
ma20 = prices.rolling(window=20).mean()                         # step 2: 20-day moving average
today_ma, week_ago_ma = ma20.iloc[-1], ma20.iloc[-6]            # step 3: today vs 5 trading days earlier
trend = "Uptrend" if today_ma > week_ago_ma else "Downtrend"    # step 4: label it
print(f"MA20 today={today_ma:.2f}, a week ago={week_ago_ma:.2f} → {trend}")
```

NOT EVERY PLAN STEP NEEDS AN LLM

Steps 1–4 are pure arithmetic. In a real system, let the LLM *write the plan and explain the result*, and let plain code (or a calculator tool) do the maths — cheaper, faster and never wrong about a sum. Deciding which steps get an LLM and which get code is called **workflow engineering**. `.iloc[-6]` is "6th row from the end" = 5 trading days before today.

Lab 2D · ReAct with LlamaIndex: the live price, with visible thinking

Same goal as Lab 1C, different framework. Here you can *watch* the Thought → Action → Observation loop as it streams.

Python · lab_2d_react_price.py · step 1 (pip install llama-index yfinance)

```python
from llama_index.core.agent.workflow import ReActAgent, AgentStream
from llama_index.core.tools import FunctionTool
from llama_index.llms.openai import OpenAI
import yfinance as yf

def get_latest_price(ticker_symbol: str):
    """Fetch the latest price for the given ticker symbol."""
    ticker = yf.Ticker(ticker_symbol)
    data = ticker.history(period="1d", interval="1m")         # today, one row per MINUTE
    if not data.empty:                                          # did any rows come back?
        latest_row = data.iloc[-1]                              # the most recent minute
        return {
            "ticker": ticker_symbol,
            "datetime": latest_row.name.strftime("%Y-%m-%d %H:%M"),   # .name = this row's label = its tim
            "price": float(latest_row["Close"]),
        }
    else:
        return {"error": f"No data found for ticker: {ticker_symbol}"}  # the agent can read this and reac

tools = [FunctionTool.from_defaults(get_latest_price)]       # LlamaIndex's way to wrap a function as a to

agent_prices = ReActAgent(
    llm=OpenAI(model="gpt-4o", api_key=OPENAI_API_KEY),
    tools=tools,
    timeout=120,              # give up after 120 s instead of hanging forever
    verbose=False,
)
```

1. `interval="1m"` asks for minute-by-minute rows instead of one row per day — so the "latest" really is the latest minute.
2. `data.iloc[-1]` is the last *row*. A single row knows its own label through `.name` — and since the rows are labelled by time, `latest_row.name` is the timestamp. (This is the confusing line. `.name` here has nothing to do with the stock's name.)
3. `.strftime("%Y-%m-%d %H:%M")` formats that timestamp as text like `2025-09-12 15:59`.
4. The `else` branch returns an **error message instead of crashing**. A ReAct agent can read "No data found" and try again with a corrected ticker.
5. `FunctionTool.from_defaults(fn)` is LlamaIndex's equivalent of `@function_tool`: it reads the name, docstring and type hints.

Python · lab_2d_react_price.py · step 2 · run it and stream the thinking Copy

```python
handler = agent_prices.run("What is the last price of Apple?")   # start the job, get a tracking handle

async for ev in handler.stream_events():        # live updates while the agent works
    if isinstance(ev, AgentStream):             # only the "text chunk" events
        print(ev.delta, end="", flush=True)     # print each new chunk on the same line, immediately

response = await handler                        # wait for the final, complete answer
```

🧠 **ANALOGY — FOOD-DELIVERY TRACKING**

`agent_prices.run(...)` places the order and hands you a **tracking link** (`handler`). `stream_events()` is the live map ("rider picked up the order..."). `AgentStream` events are the text updates — other event types (like tool calls) are skipped by the `if`. `ev.delta` is just the newest bit of text. Finally, `await handler` is the food arriving: the complete answer.

```
Thought: The current language of the user is: English. I need to use a tool to help me answer the
question.
Action: get_latest_price
Action Input: {"ticker_symbol": "AAPL"}
Thought: I can answer without using any more tools. I'll use the user's language to answer.
Answer: The latest price of Apple (AAPL) is $234.06 as of September 12, 2025, 15:59.
```

The *Observation* (the tool's result) is fed back silently between the two thoughts. Because every step is written out, you can see exactly where a wrong answer came from — very useful when an auditor asks "why did the system say that?"

	LAB 1C · OPENAI AGENTS SDK	LAB 2D · LLAMAINDEX
Make a tool	`@function_tool` above the function	`FunctionTool.from_defaults(fn)`
Make the agent	`Agent(name, instructions, model, tools)`	`ReActAgent(llm, tools, timeout)`
Run it	`Runner.run_sync(...)` → `.final_output`	`handler = agent.run(...)` → `await handler`
See the thinking	Traces / logs	Streamed Thought / Action lines

Self-consistency: let five analysts vote on an earnings surprise

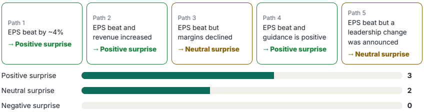
💰 **FINANCE CORNER — WHAT IS AN "EARNINGS SURPRISE"?**

Before results, analysts publish forecasts; their average EPS forecast is the **consensus**. If the company reports more, that's a *beat*. Here consensus was **1.20** and reported EPS was **1.25**: a beat of (1.25 − 1.20) ÷ 1.20 ≈ **4.2%**. But the market judges the whole picture — revenue growth, margins, guidance, management changes. An EPS beat with falling margins and a departing CEO can be a *neutral* surprise.

Self-consistency asks the model the same chain-of-thought question several times (with some randomness), then takes the **majority vote**. Each path focuses on different evidence:

🧪 **SIMULATION**

Click a path to flip its conclusion, and watch the vote



Path 1
EPS beat by ~4%
→ Positive surprise

Path 2
EPS beat and revenue increased
→ Positive surprise

Path 3
EPS beat but margins declined
→ Neutral surprise

Path 4
EPS beat and guidance is positive
→ Positive surprise

Path 5
EPS beat but a leadership change was announced
→ Neutral surprise

Positive surprise	3
Neutral surprise	2
Negative surprise	0

Majority vote → **Positive surprise** (3 of 5 paths). One brittle line of reasoning can't decide alone.

Python · self_consistency.py · try it yourself Copy

```python
from collections import Counter
from openai import OpenAI

client = OpenAI()

cot_prompt = """You are a financial analysis agent.
Classify the earnings result as: Positive surprise, Neutral surprise or Negative surprise.
Reason step by step before producing the final classification.

Input:
EPS: 1.25 vs consensus 1.20
Revenue: +4% YoY
Margins: Declining
Guidance: Raised
Management: CEO departure announced

End with exactly one line: FINAL: Positive surprise | Neutral surprise | Negative surprise"""

answers = []
for _ in range(5):                                              # 5 independent reasoning paths
    r = client.responses.create(model="gpt-4.1-mini", input=cot_prompt, temperature=1.0)
    last_line = r.output_text.strip().splitlines()[-1]          # e.g. "FINAL: Positive surprise"
    answers.append(last_line.replace("FINAL:", "").strip())

votes = Counter(answers)                                        # counts each distinct answer
print(votes)                                                    # e.g. Counter({'Positive surprise': 3, 'Neu
print("Majority:", votes.most_common(1)[0][0])
```

1. `temperature=1.0` adds randomness, so the five paths reason differently. With temperature 0 you'd get five near-identical answers and the vote would be pointless.
2. Asking for a fixed last line (`FINAL: …`) makes the answer easy to pull out with `.splitlines()[-1]`.
3. `Counter` tallies the answers; `.most_common(1)[0][0]` is "the top answer's text".

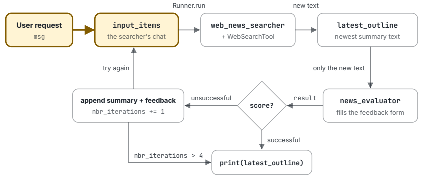
Cost note: five paths = five times the tokens. Use it for decisions that matter, not for every headline.

2.6 Lab 2E · Verified news agent (evaluator–optimizer loop)

The request: "Give me the latest 5 financial news items for the Eurozone. Reuters only, from the last two days, and a link for every item — I won't trade on anything I can't verify."

WHY SO STRICT?

News moves prices within minutes. A hallucinated headline, or a week-old story presented as today's, can trigger a wrong trade. So the bank demands a trusted source, a date window and a link per item — and a second agent that checks all three before anything reaches a trader.



The searcher keeps the whole conversation (`input_items`); the evaluator only ever sees the newest summary (`latest_outline`) — fresh eyes, fewer tokens. The loop ends on "successful" or when the retry counter passes the limit.

SIMULATION

Step through two rounds of the loop (the map above lights up)

Get the flow first; the code for every box follows right after.

◀ Back **Next step ▶** Step 1 of 6

Start

round	len(input_items)	nbr_iterations	result.score
–	1	0	–

Your request becomes the first item in the searcher's chat: `[{"content": msg, "role": "user"}]`. `latest_outline` is still `None`.

* Web-search call items are also added to `input_items`; we leave them out of the count to keep it simple.

Step 1 — Dates, so the window is always "the last two days"

Python · lab_2e_news_loop.py · step 1 (pip install openai-agents) Copy

```python
from datetime import datetime, timedelta
from dataclasses import dataclass
from typing import Literal
from agents import Agent, Runner, WebSearchTool, ItemHelpers, TResponseInputItem

today_date   = datetime.now().strftime("%Y-%m-%d")                         # e.g. "2026-10-22"
two_days_ago = (datetime.now() - timedelta(days=2)).strftime("%Y-%m-%d")   # e.g. "2026-10-20"
```

⚠ **CLASSROOM GOTCHA**

These dates are baked into the agents' instructions with f-strings at the moment you run the cell. Leave the notebook open overnight and the "last two days" window is stale — re-run this cell and the agent definitions.

Step 2 — The searcher

Python · lab_2e_news_loop.py · step 2 Copy

```python
INSTRUCTIONS_NEWS_SEARCH = (
    "You are a research news assistant specialized in Finance or any news that can have an impact "
    "on the financial health of a given company or a region. "
    "You search the web for a given region to collect the most important and recent news. "
    "Produce a concise summary of the results with citations. The summary must be "
    "2-3 paragraphs and less than 300 words. Capture the main points. Write succinctly. "
    f"Get only the latest news between 2 days ago {two_days_ago} and today {today_date}. "
    "Use only this resource: https://www.reuters.com/. "
    "For each item you are providing, specify the date on which the article was published. "
    "For each item you are providing, specify the complete external link to the article. "
    "This is important. Otherwise, don't give me a summary, if you don't provide a date and an external li
    "If you don't give a source link, it means that you are hallucinating. "
)

web_news_searcher = Agent(
    name="web_news_searcher",
    instructions=INSTRUCTIONS_NEWS_SEARCH,
    tools=[WebSearchTool()],          # OpenAI's hosted web search — no code of your own needed
)
```

Three finance constraints are encoded right in the instructions: **one trusted source**, **a date window**, and **a date + link per item**. Python joins adjacent strings in brackets into one long string, which is why it's written as many short lines.

Step 3 — The evaluator and its "form"

Python · lab_2e_news_loop.py · step 3 Copy

```python
@dataclass
class EvaluationFeedback:
    feedback: str                                    # free text: why it passed or failed
    score: Literal["successful", "unsuccessful"]     # exactly one of these two words — nothing else

news_evaluator = Agent(
    name="news_evaluator",
    instructions=(
        "You will be given a financial news summary in a given region (Euro, US, Asia) and you are "
        "asked to evaluate the completeness of external source links. "
        "Check whether the news summary is supported by external article links from https://www.reuters.co
        f"Check whether the news summary was published between 2 days ago {two_days_ago} and today {today_
        "For each item of news, the date of the news must be explicitly provided. "
        "Check whether the news summary corresponds to the region or regions requested by the user. "
        "If the user requested 'n' news items, each item must be supported by an external source link, "
        "so you need to ensure that there are 'n' source links. "
        "External links are required for the user to verify the accuracy of the news. "
    ),
    output_type=EvaluationFeedback,                  # forces the answer into the form above
)
```

> 🧠 **ANALOGY — A FORM WITH TICK-BOXES**
>
> Without `output_type`, the judge could reply "mostly fine, one link looks odd" and your `if` statement wouldn't know what to do. The dataclass is a printed form: one comment box (`feedback`) and one tick-box with exactly two options (`score`). `Literal[...]` means "only these exact values". Now `result.score == "successful"` is always a clean yes/no.

Step 4 — The loop (with comments on every line)

Python · lab_2e_news_loop.py · step 4 Copy

```python
msg = input("User's request:")    # e.g. "Provide the latest 5 news in the Eurozone and specify the source
input_items: list[TResponseInputItem] = [{"content": msg, "role": "user"}]   # the searcher's running chat
latest_outline: str | None = None   # will hold the newest news summary
max_iteration = 4                   # retry limit
nbr_iterations = 0                  # retries so far

while True:
    # 1) The searcher writes (or rewrites) the summary, seeing the whole chat so far
    news_searcher_result = await Runner.run(web_news_searcher, input_items)
    input_items = news_searcher_result.to_input_list()                       # old chat + the new reply
    latest_outline = ItemHelpers.text_message_outputs(news_searcher_result.new_items)  # just the new text

    # 2) The evaluator grades ONLY the newest summary
    news_evaluator_result = await Runner.run(news_evaluator, latest_outline)
    result: EvaluationFeedback = news_evaluator_result.final_output         # the filled-in form

    # 3) Passed? Stop.
    if result.score == "successful":
        print("Links check is good ==> stopping the iteration.")
        break

    # 4) Failed: add the summary and the judge's feedback to the chat, then go round again
    input_items.append({"content": f"News Summary: {latest_outline}", "role": "system"})
    input_items.append({"content": f"Feedback: {result.feedback}\n for each item in the news summary, "
                               "provide an external source link:", "role": "user"})
    nbr_iterations += 1
    if nbr_iterations > max_iteration:
        print("Reached max_iteration ==> stopping the iteration.")
        break

print(latest_outline)
```

VARIABLE	WHAT IT IS	CHANGES HOW?

`input_items`	The searcher's whole conversation	Grows every round (reply + summary + feedback)
`.to_input_list()`	"Everything so far, ready to feed back in"	Old input + this run's new items
`.new_items`	Only what this run produced	Fresh each round
`latest_outline`	Plain text of the newest summary	Replaced each round
`result`	The judge's form: `.score` + `.feedback`	Replaced each round
`nbr_iterations`	Retry counter	+1 after each failure

> 💡 **HOW MANY SEARCHES CAN ACTUALLY HAPPEN?**
>
> The check is `nbr_iterations > max_iteration`, so with `max_iteration = 4` the searcher can run **5 times**: the first try plus 4 retries. Change it to `≥` if you want 4 runs in total. Either way, a limit is essential — a loop between two LLMs with no limit can burn money forever.

Guardrails, finance edition

GUARDRAIL	CHECKS...	FINANCE EXAMPLE
Input	The request, before the agent starts	A fundamental-analysis agent is asked for "Tesla's 5-year price chart" → stop early instead of wasting tokens on a wrong-scope answer
Output	The answer, before the user sees it	Block any reply that contains another client's name or account details
Tool	The arguments the LLM wants to pass to a tool	Ticker must look like a ticker and must not be on the bank's restricted list before calling the data API
Task	An intermediate result, before the next step uses it	In Lab 2F, check that the retrieval JSON really contains numbers before the analysis agent runs

Framework support differs: the OpenAI Agents SDK has input/output guardrails, Google ADK has tool guardrails, CrewAI has task guardrails.

2.7 Lab 2F · Buy / Hold / Sell: three agents, one JSON baton

The request: "Give me a one-glance view on Apple: Buy, Hold or Sell — with the reasons and what would make you change your mind."

SIMULATION

Click each agent to see what it adds to the baton

1 · Retrieval
calls the ratio tool; facts only

→ JSON

2 · Analysis
rates profitability & valuation, lists risks

→ JSON

3 · Decision
BUY / HOLD / SELL + watchlist

Each arrow is one line of code: the previous run's `.final_output` (a JSON string) becomes the next agent's input.

Agent 1 output — Adds: raw numbers only. No opinions — that's the analyst's job.

```
{
  "ticker": "AAPL",
  "fundamentals": {
    "profitability_ratios": { "2025": { "Gross Margin": 0.4691, "Operating Margin": 0.3197,
        "Net Profit Margin": 0.2692, "Return on Assets": 0.3093, "Return on Equity": 1.7142 } },
    "valuation_ratios": { "2025": { "Price-to-Earnings": 36.418, "Price-to-Book": 55.3236,
        "Price-to-Cash-Flow": 36.5905, "Price-to-Free-Cash-Flow": 41.301,
        "EV-to-Sales": 9.9914, "EV-to-EBITDA": 28.7259 } }
  }
}
```

> **ANALOGY — A RELAY RACE**
>
> Each runner does one leg and hands over a baton. Here the baton is a JSON string. Runner 1 must not interpret ("no interpretation" is in its instructions), runner 2 must not fetch data, runner 3 decides *only* from what it is handed. That separation is what makes the final decision auditable.

Step 1 — The tool (profitability + valuation only)

Same function as Lab 2A, but the liquidity list is gone: a long-term Buy/Hold/Sell call is about business quality and price, so we keep the context small and focused.

Python · lab_2f_buy_hold_sell.py · step 1

Copy

```python
@function_tool
def get_company_fundamentals(ticker: str) → dict[str, pd.DataFrame]:
    """Get the profitability and valuation fundamental ratios for a given ticker."""
    companies = Toolkit([ticker], api_key=FINANCIAL_MODELING_PREP_API_KEY, start_date="2022-01-01")
    ratios = companies.ratios.collect_all_ratios()
    profitability_ratios_names = ["Return on Equity", "Return on Assets", "Gross Margin",
                                  "Operating Margin", "Net Profit Margin"]
    valuation_ratios_names = ["Price-to-Earnings", "Price-to-Book", "Price-to-Cash-Flow",
                              "Price-to-Free-Cash-Flow", "EV-to-EBITDA", "EV-to-Sales"]
    return {
        "profitability_ratios": ratios.loc[ratios.index.intersection(profitability_ratios_names)],
        "valuation_ratios":     ratios.loc[ratios.index.intersection(valuation_ratios_names)],
    }
```

Step 2 — Three agents, each with a JSON contract

Python · lab_2f_buy_hold_sell.py · step 2 Copy

```python
MODEL = "gpt-4o-mini"

fundamental_retrieval_agent = Agent(
    name="Fundamental Retrieval Agent", model=MODEL, tools=[get_company_fundamentals],
    instructions="""
You are the Fundamental Retrieval Agent.
Tasks:
1) Call the fundamentals tool for the specified ticker.
2) Output ONLY valid JSON with this schema. Keep the output strictly factual: no interpretation.
{ "ticker": "...", "fundamentals": { ... } }   // raw numbers from tools
""",
)

fundamental_analysis_agent = Agent(
    name="Fundamental Analysis Agent", model=MODEL,
    instructions="""
You are the Fundamental Analysis Agent.
Input: JSON from the Retrieval Agent.
Tasks:
1) Interpret the fundamentals along 2 dimensions:
   - Profitability (profitability, margins, ROE)
   - Valuation (P/E, EV/Sales)
2) Identify 2-3 key risks (e.g., valuation risk, margin sustainability).
3) Output ONLY valid JSON with this schema:
{
  "ticker": "...",
  "fundamentals": { ... },          // keep from input
  "analysis": {
    "Profitability": { "rating": "strong|neutral|weak", "evidence": [string, ...] },
    "valuation": { "rating": "cheap|fair|expensive", "evidence": [string, ...] },
    "risks": [string, ...]
  }
}
Rules:
- Keep evidence short and specific.
""",
)

decision_agent = Agent(
    name="Decision Agent", model=MODEL,
    instructions="""
You are the Decision Agent.
Input: JSON from the Fundamental Analysis Agent.
Tasks: Based only on the input coming from the Fundamental Analysis Agent:
1) Produce a simple stance: BUY, HOLD, or SELL.
2) Provide 3 bullet reasons grounded in the analysis.
3) Provide a "watchlist" section: what metrics would change your stance.
Output ONLY valid JSON:
{ "ticker": "...", "stance": "BUY|HOLD|SELL",
  "summary": { "reasons": [string, string, string], "watchlist": [string, string, string] } }
""",
)
```

① Each agent's instructions end with the **exact JSON shape** it must hand on — the contract between relay runners.

② `"rating": "strong|neutral|weak"` means "pick one of these words". Fixed vocabularies keep the next agent's job simple.

③ The analysis agent copies `fundamentals` forward ("keep from input"), so the decision agent can still see the raw numbers.

Step 3 — The whole orchestration is three lines

Python · lab_2f_buy_hold_sell.py · step 3 Copy

```python
fundamental_results  = await Runner.run(fundamental_retrieval_agent, "Provide Apple fundamentals")
fundamental_analysis = await Runner.run(fundamental_analysis_agent, fundamental_results.final_output)
final_decision       = await Runner.run(decision_agent, fundamental_analysis.final_output)
print(final_decision.final_output)
```

Read it as a relay: the `.final_output` (a JSON string) of one run becomes the `input` of the next. No framework magic.

💰 **READING THE VERDICT: HOLD**

Profitability: strong (margins above 30%, net margin ~25–27%). **Valuation: expensive** — the analysis noted P/E climbing from about 21 to about 41 over three years, meaning the share price rose much faster than profits. Great business, pricey stock → **HOLD**: don't sell a winner, don't overpay to buy more.

The **watchlist** is the most useful part for a PM: tripwires that would flip the call — "P/E or price-to-book falls", "net profit margin drops below 25%". It turns a one-off opinion into a monitoring rule.

⚠️ **SPOT THE LEAK**

One listed risk was "market competition impacting future growth". No ratio says that — the LLM added it from its own general knowledge. In a regulated memo every claim should trace to evidence. Fix: add "every risk must cite a ratio from the input" to the analysis instructions, or ground it with retrieved industry reports.

Python · a task guardrail on the final baton (optional add-on)

```python
import json

decision = json.loads(final_decision.final_output)           # crashes here = the agent broke the JSON co
assert decision["stance"] in {"BUY", "HOLD", "SELL"}, "Unexpected stance!"
assert len(decision["summary"]["watchlist"]) == 3, "Watchlist must have 3 items"
```

If a model ever wraps its JSON in a ``` code fence, `json.loads` fails. The robust fix is the one from Lab 2E: give the agent an `output_type` (a dataclass or Pydantic model) instead of describing JSON in words.

> **DISCLAIMER**
>
> Everything in this lab is for learning. It is not investment advice or a recommendation to buy or sell any security.

2.8 Context and memory upkeep — the finance notes

You know context engineering. In a rebalancing step, a good context holds: the **current allocation**, **recent volatility** from the bank's database, **relevant past decisions** from long-term memory, and the **description of a pricing tool** — not ten years of trade history. Two framework behaviours worth knowing:

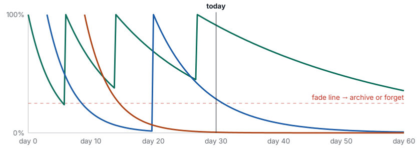
- **LlamaIndex** can flush short-term memory into long-term memory when chat history crosses a token threshold (e.g. 40K tokens), into three blocks: *static* (fixed facts like the client's mandate), *fact extraction* (facts pulled out of the chat) and *vector* (similarity search over past turns).
- **CrewAI** detects when the context overflows and either summarises it or stops with an error, depending on settings.

For long-term memory, two upkeep ideas: **hierarchical summaries** (detailed chats → daily summaries → an overall summary → a short client profile) and **forgetting**, where unused memories fade and recalled ones get stronger — modelled on the human forgetting curve.

SIMULATION

Memory strength over time — press "Recall" to reinforce a memory

Today is day 30



🟩 Client: no Energy exposure: **91%** strong

🟦 High-volatility playbook: **29%** fading

🟥 One-off gold-ETF question: **1%** faded

Each recall resets the memory to 100% *and* doubles how slowly it fades next time. Frequently used client preferences stay; one-off trivia fades out of retrieval.

> **COMPLIANCE NOTE ON CLIENT PROFILES**
>
> Memory frameworks can build a "user portrait" automatically. In finance, keep it **explicit and correctable** — "max 35% per sector, set by the client on 12 Oct" — never an inferred guess about the client's mood or personality. Suitability rules require you to show where every client preference came from, and let the client change it.

WRAP-UP

What to remember

1. **Pick models by data sensitivity first, reasoning depth second.** Confidential data pushes you to self-hosted open models; deep analysis pushes you to reasoning models.
2. **Valid JSON isn't usable JSON.** Name every key, type and unit — or use structured outputs.
3. **Tools make numbers trustworthy; interpretation is still the LLM's.** Know your ratios well enough to challenge it (ROE 171%, current ratio 0.89).
4. **Working memory = the session notepad.** Pass the same `session` every turn or the agent forgets the portfolio.
5. **Episodic memory = journal → cabinet → retrieve → paste into the prompt.** Follow the variables: `conversation_history` → string → prompt → `episodic_memory` → Chroma → `['documents'][0][0]`.

⑥ **Checks need limits and contracts.** A judge with a fixed form (`output_type`), a retry limit, and JSON contracts between agents make a system auditable.

Quiz Check yourself

1. Apple's return on equity is 171%. What's the most likely reason?

- Apple is a very risky company
- Buybacks have made shareholders' equity small, so profit ÷ equity is huge
- The data API made an error
- Revenue doubled last year

2. Turn 2 of the rebalancing chat is run WITHOUT the session. What goes wrong?

- The agent doesn't know the 70/20/10 starting point, so it can't size the trades
- Nothing — the agent still remembers
- The API call fails with an error
- The answer is slower

3. In episodic_memory_recap['documents'][0][0], what does the FIRST [0] pick?

- The best match
- The first memory ever stored
- The first word of the document
- The first question you passed in query_texts

4. Why does the prompt template write {{ and }} around the JSON example?

- It's a typo
- JSON needs double braces
- So .format() leaves a literal { } instead of treating it as a slot
- To make the LLM think harder

5. With max_iteration = 4 and the check nbr_iterations > max_iteration, how many times can the news searcher run?

- 3
- 4
- 5
- Forever

6. You must tag 50,000 public headlines with sentiment every hour. Best fit?

- A fast non-reasoning model or a FinBERT-style classifier
- A hosted reasoning model at high effort
- Fine-tune a 70B model first
- A multi-agent pipeline

Answer all six to see your score.

For learning only — nothing here is investment advice. Sample outputs vary run to run; prices, ratios and news are live.